

**Akash Rajan STK-26-5266 (3 to 4)Batch**

## **MySQL Assignment — Answer Document**

Banking, Railway Reservation, Employee Management,  
Customers & Orders, and Students Scenarios

## BANKING SCENARIO — Basic + WHERE + Operators (Q1–15)

**Q1. Retrieve all accounts with balance greater than 20,000.**

```
SELECT * FROM accounts WHERE balance > 20000;
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 1          | 1           | Savings      | 50000.00 |
| 3          | 2           | Savings      | 30000.00 |
| 5          | 4           | Current      | 40000.00 |

**Q2. Find customers who live in Chennai.**

```
SELECT * FROM customers WHERE city = 'Chennai';
```

| customer_id | name   | city    |
|-------------|--------|---------|
| 1           | Janani | Chennai |

**Q3. Display accounts with balance between 20,000 and 50,000.**

```
SELECT * FROM accounts WHERE balance BETWEEN 20000 AND 50000;
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 1          | 1           | Savings      | 50000.00 |
| 2          | 1           | Current      | 20000.00 |
| 3          | 2           | Savings      | 30000.00 |
| 5          | 4           | Current      | 40000.00 |

**Q4. Find customers whose names start with 'J'.**

```
SELECT * FROM customers WHERE name LIKE 'J%';
```

| customer_id | name   | city    |
|-------------|--------|---------|
| 1           | Janani | Chennai |

**Q5. Retrieve accounts of type 'Savings' or 'Current'.**

```
SELECT * FROM accounts WHERE account_type IN ('Savings','Current');
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 1          | 1           | Savings      | 50000.00 |
| 2          | 1           | Current      | 20000.00 |
| 3          | 2           | Savings      | 30000.00 |
| 4          | 3           | Savings      | 15000.00 |
| 5          | 4           | Current      | 40000.00 |

**Q6. Display accounts that are not 'Savings'.**

```
SELECT * FROM accounts WHERE account_type <> 'Savings';
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 2          | 1           | Current      | 20000.00 |
| 5          | 4           | Current      | 40000.00 |

**Q7. Find customers whose names contain the letter 'a'.**

```
SELECT * FROM customers WHERE name LIKE '%a%';
```

| customer_id | name    | city       |
|-------------|---------|------------|
| 1           | Janani  | Chennai    |
| 2           | Arun    | Coimbatore |
| 3           | Priya   | Madurai    |
| 4           | Karthik | Salem      |

**Q8. Retrieve accounts with balance less than or equal to 30,000.**

```
SELECT * FROM accounts WHERE balance <= 30000;
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 2          | 1           | Current      | 20000.00 |

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 3          | 2           | Savings      | 30000.00 |
| 4          | 3           | Savings      | 15000.00 |

**Q9. Find customers who are not from Madurai.**

```
SELECT * FROM customers WHERE city <> 'Madurai';
```

| customer_id | name    | city       |
|-------------|---------|------------|
| 1           | Janani  | Chennai    |
| 2           | Arun    | Coimbatore |
| 4           | Karthik | Salem      |

**Q10. Display accounts where balance is not between 10,000 and 40,000.**

```
SELECT * FROM accounts WHERE balance NOT BETWEEN 10000 AND 40000;
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 1          | 1           | Savings      | 50000.00 |

**Q11. Retrieve customers whose names end with 'i'.**

```
SELECT * FROM customers WHERE name LIKE '%i';
```

| customer_id | name   | city    |
|-------------|--------|---------|
| 1           | Janani | Chennai |

**Q12. Find accounts with balance equal to 50,000.**

```
SELECT * FROM accounts WHERE balance = 50000;
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 1          | 1           | Savings      | 50000.00 |

**Q13. Display customers whose city is either Chennai or Salem.**

```
SELECT * FROM customers WHERE city IN ('Chennai','Salem');
```

| customer_id | name    | city    |
|-------------|---------|---------|
| 1           | Janani  | Chennai |
| 4           | Karthik | Salem   |

**Q14. Find accounts with balance greater than 10,000 and less than 40,000.**

```
SELECT * FROM accounts WHERE balance > 10000 AND balance < 40000;
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 2          | 1           | Current      | 20000.00 |
| 3          | 2           | Savings      | 30000.00 |
| 4          | 3           | Savings      | 15000.00 |

**Q15. Retrieve accounts where account type is not in ('Current').**

```
SELECT * FROM accounts WHERE account_type NOT IN ('Current');
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 1          | 1           | Savings      | 50000.00 |
| 3          | 2           | Savings      | 30000.00 |
| 4          | 3           | Savings      | 15000.00 |

## BANKING SCENARIO — ORDER BY (Q16–18)

**Q16. Display all accounts sorted by balance in descending order.**

```
SELECT * FROM accounts ORDER BY balance DESC;
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 1          | 1           | Savings      | 50000.00 |
| 5          | 4           | Current      | 40000.00 |
| 3          | 2           | Savings      | 30000.00 |
| 2          | 1           | Current      | 20000.00 |
| 4          | 3           | Savings      | 15000.00 |

**Q17. List customers sorted alphabetically by name.**

```
SELECT * FROM customers ORDER BY name ASC;
```

| customer_id | name    | city       |
|-------------|---------|------------|
| 2           | Arun    | Coimbatore |
| 1           | Janani  | Chennai    |
| 4           | Karthik | Salem      |
| 3           | Priya   | Madurai    |

**Q18. Display accounts sorted by account type and then by balance (descending).**

```
SELECT * FROM accounts ORDER BY account_type ASC, balance DESC;
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 5          | 4           | Current      | 40000.00 |
| 2          | 1           | Current      | 20000.00 |
| 1          | 1           | Savings      | 50000.00 |
| 3          | 2           | Savings      | 30000.00 |
| 4          | 3           | Savings      | 15000.00 |

# BANKING SCENARIO — Aggregate Functions (Q19–23)

Q19. Find the total balance of all accounts.

|                                                     |  |
|-----------------------------------------------------|--|
| SELECT SUM(balance) AS total_balance FROM accounts; |  |
| total_balance                                       |  |
| 155000.00                                           |  |

Q20. Calculate the average balance of accounts.

|                                                       |  |
|-------------------------------------------------------|--|
| SELECT AVG(balance) AS average_balance FROM accounts; |  |
| average_balance                                       |  |
| 31000.000000                                          |  |

Q21. Find the maximum account balance.

|                                                   |  |
|---------------------------------------------------|--|
| SELECT MAX(balance) AS max_balance FROM accounts; |  |
| max_balance                                       |  |
| 50000.00                                          |  |

Q22. Find the minimum account balance.

|                                                   |  |
|---------------------------------------------------|--|
| SELECT MIN(balance) AS min_balance FROM accounts; |  |
| min_balance                                       |  |
| 15000.00                                          |  |

Q23. Count the total number of customers.

|                                                    |  |
|----------------------------------------------------|--|
| SELECT COUNT(*) AS total_customers FROM customers; |  |
| total_customers                                    |  |
| 4                                                  |  |

## BANKING SCENARIO — GROUP BY + HAVING (Q24–28)

**Q24. Find total balance grouped by account type.**

```
SELECT account_type, SUM(balance) AS total_balance FROM accounts GROUP BY account_type;
```

| account_type | total_balance |
|--------------|---------------|
| Savings      | 95000.00      |
| Current      | 60000.00      |

**Q25. Find average balance for each account type.**

```
SELECT account_type, AVG(balance) AS avg_balance FROM accounts GROUP BY account_type;
```

| account_type | avg_balance  |
|--------------|--------------|
| Savings      | 31666.666667 |
| Current      | 30000.000000 |

**Q26. Display account types having average balance greater than 20,000.**

```
SELECT account_type, AVG(balance) AS avg_balance FROM accounts GROUP BY account_type HAVING AVG(balance) > 20000;
```

| account_type | avg_balance  |
|--------------|--------------|
| Savings      | 31666.666667 |
| Current      | 30000.000000 |

**Q27. Count number of accounts for each customer.**

```
SELECT customer_id, COUNT(*) AS account_count FROM accounts GROUP BY customer_id;
```

| customer_id | account_count |
|-------------|---------------|
| 1           | 2             |
| 2           | 1             |
| 3           | 1             |
| 4           | 1             |

**Q28. Display customers having more than one account.**

```
SELECT customer_id, COUNT(*) AS account_count FROM accounts GROUP BY customer_id HAVING COUNT(*) > 1;
```

| customer_id | account_count |
|-------------|---------------|
| 1           | 2             |

## BANKING SCENARIO — JOINS (Q29–35)

**Q29. Retrieve customer names along with their account balances.**

```
SELECT c.name, a.balance FROM customers c JOIN accounts a ON c.customer_id = a.customer_id;
```

| name    | balance  |
|---------|----------|
| Janani  | 50000.00 |
| Janani  | 20000.00 |
| Arun    | 30000.00 |
| Priya   | 15000.00 |
| Karthik | 40000.00 |

**Q30. Display all customers and their accounts (including customers without accounts).**

```
SELECT c.name, a.account_type, a.balance FROM customers c LEFT JOIN accounts a ON c.customer_id = a.customer_id;
```

| name    | account_type | balance  |
|---------|--------------|----------|
| Janani  | Savings      | 50000.00 |
| Janani  | Current      | 20000.00 |
| Arun    | Savings      | 30000.00 |
| Priya   | Savings      | 15000.00 |
| Karthik | Current      | 40000.00 |

**Q31. Display all accounts and corresponding customer details.**

```
SELECT a.account_id, a.account_type, a.balance, c.name, c.city FROM accounts a JOIN customers c ON a.customer_id = c.customer_id;
```

| account_id | account_type | balance  | name    | city       |
|------------|--------------|----------|---------|------------|
| 1          | Savings      | 50000.00 | Janani  | Chennai    |
| 2          | Current      | 20000.00 | Janani  | Chennai    |
| 3          | Savings      | 30000.00 | Arun    | Coimbatore |
| 4          | Savings      | 15000.00 | Priya   | Madurai    |
| 5          | Current      | 40000.00 | Karthik | Salem      |

**Q32. Retrieve customer names and account types where balance is greater than 20,000.**

```
SELECT c.name, a.account_type, a.balance FROM customers c JOIN accounts a ON c.customer_id = a.customer_id WHERE a.balance > 20000;
```

| name    | account_type | balance  |
|---------|--------------|----------|
| Janani  | Savings      | 50000.00 |
| Arun    | Savings      | 30000.00 |
| Karthik | Current      | 40000.00 |

**Q33. List customers with their total balance using JOIN.**

```
SELECT c.name, SUM(a.balance) AS total_balance FROM customers c JOIN accounts a ON c.customer_id = a.customer_id GROUP BY c.name;
```

| name    | total_balance |
|---------|---------------|
| Janani  | 70000.00      |
| Arun    | 30000.00      |
| Priya   | 15000.00      |
| Karthik | 40000.00      |

**Q34. Display customer names and balances sorted by balance.**

```
SELECT c.name, a.balance FROM customers c JOIN accounts a ON c.customer_id = a.customer_id ORDER BY a.balance DESC;
```

| name    | balance  |
|---------|----------|
| Janani  | 50000.00 |
| Karthik | 40000.00 |
| Arun    | 30000.00 |

| name   | balance  |
|--------|----------|
| Janani | 20000.00 |
| Priya  | 15000.00 |

**Q35. Count number of accounts for each city using JOIN.**

```
SELECT c.city, COUNT(a.account_id) AS account_count FROM customers c JOIN accounts a ON c.customer_id = a.customer_id GROUP BY c.city;
```

| city       | account_count |
|------------|---------------|
| Chennai    | 2             |
| Coimbatore | 1             |
| Madurai    | 1             |
| Salem      | 1             |

## BANKING SCENARIO — SUBQUERIES (Q36–40)

**Q36. Find accounts with balance greater than average balance.**

```
SELECT * FROM accounts WHERE balance > (SELECT AVG(balance) FROM accounts);
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 1          | 1           | Savings      | 50000.00 |
| 5          | 4           | Current      | 40000.00 |

**Q37. Retrieve customers who have accounts.**

```
SELECT * FROM customers WHERE customer_id IN (SELECT customer_id FROM accounts);
```

| customer_id | name    | city       |
|-------------|---------|------------|
| 1           | Janani  | Chennai    |
| 2           | Arun    | Coimbatore |
| 3           | Priya   | Madurai    |
| 4           | Karthik | Salem      |

**Q38. Find customers who do not have any accounts.**

```
SELECT * FROM customers WHERE customer_id NOT IN (SELECT customer_id FROM accounts);
```

*(empty result set)*

**Q39. Display account(s) with the maximum balance.**

```
SELECT * FROM accounts WHERE balance = (SELECT MAX(balance) FROM accounts);
```

| account_id | customer_id | account_type | balance  |
|------------|-------------|--------------|----------|
| 1          | 1           | Savings      | 50000.00 |

**Q40. Find customers whose total balance is greater than 40,000.**

```
SELECT customer_id, SUM(balance) AS total_balance FROM accounts GROUP BY customer_id HAVING SUM(balance) > 40000;
```

| customer_id | total_balance |
|-------------|---------------|
| 1           | 70000.00      |

## RAILWAY RESERVATION — Basic + WHERE + Operators (Q41–45)

**Q41. Retrieve all bookings with fare greater than 400.**

```
SELECT * FROM bookings WHERE fare > 400;
```

| booking_id | train_id | passenger_name | fare   | status    |
|------------|----------|----------------|--------|-----------|
| 1          | 1        | Janani         | 500.00 | Confirmed |
| 2          | 1        | Arun           | 500.00 | Waiting   |
| 4          | 3        | Karthik        | 450.00 | Cancelled |

**Q42. Find bookings where status is not 'Confirmed'.**

```
SELECT * FROM bookings WHERE status <> 'Confirmed';
```

| booking_id | train_id | passenger_name | fare   | status    |
|------------|----------|----------------|--------|-----------|
| 2          | 1        | Arun           | 500.00 | Waiting   |
| 4          | 3        | Karthik        | 450.00 | Cancelled |

**Q43. Display trains starting from Chennai.**

```
SELECT * FROM trains WHERE source = 'Chennai';
```

| train_id | train_name | source  | destination |
|----------|------------|---------|-------------|
| 1        | Express1   | Chennai | Madurai     |

**Q44. Retrieve bookings with fare between 300 and 500.**

```
SELECT * FROM bookings WHERE fare BETWEEN 300 AND 500;
```

| booking_id | train_id | passenger_name | fare   | status    |
|------------|----------|----------------|--------|-----------|
| 1          | 1        | Janani         | 500.00 | Confirmed |
| 2          | 1        | Arun           | 500.00 | Waiting   |
| 3          | 2        | Priya          | 300.00 | Confirmed |
| 4          | 3        | Karthik        | 450.00 | Cancelled |
| 5          | 2        | Meena          | 300.00 | Confirmed |

**Q45. Find passengers whose names start with 'A'.**

```
SELECT * FROM bookings WHERE passenger_name LIKE 'A%';
```

| booking_id | train_id | passenger_name | fare   | status  |
|------------|----------|----------------|--------|---------|
| 2          | 1        | Arun           | 500.00 | Waiting |

## RAILWAY RESERVATION — JOINS + GROUP BY + ORDER BY (Q46–48)

**Q46. Retrieve train names along with passenger names.**

```
SELECT t.train_name, b.passenger_name FROM trains t JOIN bookings b ON t.train_id = b.train_id;
```

| train_name | passenger_name |
|------------|----------------|
| Express1   | Janani         |
| Express1   | Arun           |
| Express2   | Priya          |
| Express2   | Meena          |
| Express3   | Karthik        |

**Q47. Count number of bookings for each train.**

```
SELECT t.train_name, COUNT(b.booking_id) AS booking_count FROM trains t JOIN bookings b ON t.train_id = b.train_id GROUP BY t.train_name;
```

| train_name | booking_count |
|------------|---------------|
| Express1   | 2             |
| Express2   | 2             |
| Express3   | 1             |

**Q48. Display train names and total fare collected for each train.**

```
SELECT t.train_name, SUM(b.fare) AS total_fare FROM trains t JOIN bookings b ON t.train_id = b.train_id GROUP BY t.train_name;
```

| train_name | total_fare |
|------------|------------|
| Express1   | 1000.00    |
| Express2   | 600.00     |
| Express3   | 450.00     |

## RAILWAY RESERVATION — SUBQUERIES (Q49–50)

**Q49. Find bookings with fare equal to the highest fare.**

```
SELECT * FROM bookings WHERE fare = (SELECT MAX(fare) FROM bookings);
```

| booking_id | train_id | passenger_name | fare   | status    |
|------------|----------|----------------|--------|-----------|
| 1          | 1        | Janani         | 500.00 | Confirmed |
| 2          | 1        | Arun           | 500.00 | Waiting   |

**Q50. Retrieve trains that have more than one booking.**

```
SELECT t.train_name, COUNT(b.booking_id) AS booking_count FROM trains t JOIN bookings b ON t.train_id = b.train_id GROUP BY t.train_name HAVING COUNT(b.booking_id) > 1;
```

| train_name | booking_count |
|------------|---------------|
| Express1   | 2             |
| Express2   | 2             |

## EMPLOYEE MANAGEMENT

**Q1. Find the total number of employees in each department.**

```
SELECT department, COUNT(*) AS total_employees FROM Employee GROUP BY department;
```

| department | total_employees |
|------------|-----------------|
| IT         | 2               |
| HR         | 2               |
| Finance    | 2               |

**Q2. Find the average salary of employees in each department.**

```
SELECT department, AVG(salary) AS avg_salary FROM Employee GROUP BY department;
```

| department | avg_salary   |
|------------|--------------|
| IT         | 65000.000000 |
| HR         | 46500.000000 |
| Finance    | 60000.000000 |

**Q3. Display departments having more than one employee.**

```
SELECT department, COUNT(*) AS total_employees FROM Employee GROUP BY department HAVING COUNT(*) > 1;
```

| department | total_employees |
|------------|-----------------|
| IT         | 2               |
| HR         | 2               |
| Finance    | 2               |

**Q4. Find the highest salary in each department.**

```
SELECT department, MAX(salary) AS highest_salary FROM Employee GROUP BY department;
```

| department | highest_salary |
|------------|----------------|
| IT         | 70000.00       |
| HR         | 48000.00       |
| Finance    | 65000.00       |

**Q5. Find the lowest salary in each department.**

```
SELECT department, MIN(salary) AS lowest_salary FROM Employee GROUP BY department;
```

| department | lowest_salary |
|------------|---------------|
| IT         | 60000.00      |
| HR         | 45000.00      |
| Finance    | 55000.00      |

**Q6. Find departments whose average salary is greater than 50,000.**

```
SELECT department, AVG(salary) AS avg_salary FROM Employee GROUP BY department HAVING AVG(salary) > 50000;
```

| department | avg_salary   |
|------------|--------------|
| IT         | 65000.000000 |
| Finance    | 60000.000000 |

**Q7. Calculate the total salary expenditure for each department.**

```
SELECT department, SUM(salary) AS total_salary FROM Employee GROUP BY department;
```

| department | total_salary |
|------------|--------------|
| IT         | 130000.00    |
| HR         | 93000.00     |
| Finance    | 120000.00    |

**Q8. Display all employees sorted by salary in descending order.**

```
SELECT * FROM Employee ORDER BY salary DESC;
```

| emp_id | emp_name | department | salary   | city      | joining_date |
|--------|----------|------------|----------|-----------|--------------|
| 103    | Smith    | IT         | 70000.00 | Chennai   | 2020-07-12   |
| 106    | Linda    | Finance    | 65000.00 | Mumbai    | 2021-08-18   |
| 101    | John     | IT         | 60000.00 | Chennai   | 2022-01-15   |
| 104    | Mary     | Finance    | 55000.00 | Mumbai    | 2023-01-20   |
| 105    | James    | HR         | 48000.00 | Delhi     | 2022-05-05   |
| 102    | David    | HR         | 45000.00 | Bangalore | 2021-03-10   |

**Q9. Display employees sorted first by department and then by salary in descending order.**

```
SELECT * FROM Employee ORDER BY department ASC, salary DESC;
```

| emp_id | emp_name | department | salary   | city      | joining_date |
|--------|----------|------------|----------|-----------|--------------|
| 106    | Linda    | Finance    | 65000.00 | Mumbai    | 2021-08-18   |
| 104    | Mary     | Finance    | 55000.00 | Mumbai    | 2023-01-20   |
| 105    | James    | HR         | 48000.00 | Delhi     | 2022-05-05   |
| 102    | David    | HR         | 45000.00 | Bangalore | 2021-03-10   |
| 103    | Smith    | IT         | 70000.00 | Chennai   | 2020-07-12   |
| 101    | John     | IT         | 60000.00 | Chennai   | 2022-01-15   |

**Q10. Find cities that have more than one employee.**

```
SELECT city, COUNT(*) AS employee_count FROM Employee GROUP BY city HAVING COUNT(*) > 1;
```

| city    | employee_count |
|---------|----------------|
| Chennai | 2              |
| Mumbai  | 2              |

**Q11. Find the total salary paid in each city.**

```
SELECT city, SUM(salary) AS total_salary FROM Employee GROUP BY city;
```

| city      | total_salary |
|-----------|--------------|
| Chennai   | 130000.00    |
| Bangalore | 45000.00     |
| Mumbai    | 120000.00    |
| Delhi     | 48000.00     |

**Q12. Display departments ordered by total salary expenditure from highest to lowest.**

```
SELECT department, SUM(salary) AS total_salary FROM Employee GROUP BY department ORDER BY total_salary DESC;
```

| department | total_salary |
|------------|--------------|
| IT         | 130000.00    |
| Finance    | 120000.00    |
| HR         | 93000.00     |

**Q13. Find the number of employees in each department whose salary is greater than 50,000.**

```
SELECT department, COUNT(*) AS employee_count FROM Employee WHERE salary > 50000 GROUP BY department;
```

| department | employee_count |
|------------|----------------|
| IT         | 2              |
| Finance    | 2              |

**Q14. Find the difference between the highest and lowest salary in each department.**

```
SELECT department, MAX(salary) - MIN(salary) AS salary_difference FROM Employee GROUP BY department;
```

| department | salary_difference |
|------------|-------------------|
| IT         | 10000.00          |
| HR         | 3000.00           |
| Finance    | 10000.00          |

**Q15. Display the top 3 highest-paid employees.**

```
SELECT * FROM Employee ORDER BY salary DESC LIMIT 3;
```

| emp_id | emp_name | department | salary   | city    | joining_date |
|--------|----------|------------|----------|---------|--------------|
| 103    | Smith    | IT         | 70000.00 | Chennai | 2020-07-12   |
| 106    | Linda    | Finance    | 65000.00 | Mumbai  | 2021-08-18   |
| 101    | John     | IT         | 60000.00 | Chennai | 2022-01-15   |

## CUSTOMERS & ORDERS TABLES

**Q1. Find the total order amount for each customer.**

```
SELECT c.customer_name, SUM(o.amount) AS total_amount FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name;
```

| customer_name | total_amount |
|---------------|--------------|
| Ravi          | 8000.00      |
| Divya         | 16000.00     |
| Janu          | 12000.00     |

**Q2. Find customers who have placed more than 3 orders.**

```
SELECT c.customer_name, COUNT(o.order_id) AS order_count FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name HAVING COUNT(o.order_id) > 3;
```

**Q3. Find the average order amount for each customer.**

```
SELECT c.customer_name, AVG(o.amount) AS avg_amount FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name;
```

| customer_name | avg_amount   |
|---------------|--------------|
| Ravi          | 4000.000000  |
| Divya         | 8000.000000  |
| Janu          | 12000.000000 |

**Q4. Find the highest order amount placed by each customer.**

```
SELECT c.customer_name, MAX(o.amount) AS highest_amount FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name;
```

| customer_name | highest_amount |
|---------------|----------------|
| Ravi          | 5000.00        |
| Divya         | 9000.00        |
| Janu          | 12000.00       |

**Q5. Display customers sorted by their total purchase amount.**

```
SELECT c.customer_name, SUM(o.amount) AS total_amount FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name ORDER BY total_amount DESC;
```

| customer_name | total_amount |
|---------------|--------------|
| Divya         | 16000.00     |
| Janu          | 12000.00     |
| Ravi          | 8000.00      |

**Q6. Find customers whose total purchase amount exceeds 10,000.**

```
SELECT c.customer_name, SUM(o.amount) AS total_amount FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name HAVING SUM(o.amount) > 10000;
```

| customer_name | total_amount |
|---------------|--------------|
| Divya         | 16000.00     |
| Janu          | 12000.00     |

**Q7. Display customer names along with the total number of orders placed.**

```
SELECT c.customer_name, COUNT(o.order_id) AS total_orders FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name;
```

| customer_name | total_orders |
|---------------|--------------|
| Ravi          | 2            |
| Divya         | 2            |
| Janu          | 1            |

**Q8. Find the customer who spent the highest amount.**

```
SELECT c.customer_name, SUM(o.amount) AS total_amount FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name ORDER BY total_amount DESC LIMIT 1;
```

| customer_name | total_amount |
|---------------|--------------|
| Divya         | 16000.00     |

**Q9. Find the customer who placed the maximum number of orders.**

```
SELECT c.customer_name, COUNT(o.order_id) AS total_orders FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name ORDER BY total_orders DESC LIMIT 1;
```

| customer_name | total_orders |
|---------------|--------------|
| Ravi          | 2            |

**Q10. Find customers whose average order amount is greater than 2,000.**

```
SELECT c.customer_name, AVG(o.amount) AS avg_amount FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name HAVING AVG(o.amount) > 2000;
```

| customer_name | avg_amount   |
|---------------|--------------|
| Ravi          | 4000.000000  |
| Divya         | 8000.000000  |
| Janu          | 12000.000000 |

**Q11. Display the top 5 customers based on total purchase amount.**

```
SELECT c.customer_name, SUM(o.amount) AS total_amount FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name ORDER BY total_amount DESC LIMIT 5;
```

| customer_name | total_amount |
|---------------|--------------|
| Divya         | 16000.00     |
| Janu          | 12000.00     |
| Ravi          | 8000.00      |

**Q12. Find the minimum order amount for each customer.**

```
SELECT c.customer_name, MIN(o.amount) AS min_amount FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name;
```

| customer_name | min_amount |
|---------------|------------|
| Ravi          | 3000.00    |
| Divya         | 7000.00    |
| Janu          | 12000.00   |

**Q13. Find customers who have placed orders worth more than 5,000 in total.**

```
SELECT c.customer_name, SUM(o.amount) AS total_amount FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name HAVING SUM(o.amount) > 5000;
```

| customer_name | total_amount |
|---------------|--------------|
| Ravi          | 8000.00      |
| Divya         | 16000.00     |
| Janu          | 12000.00     |

**Q14. Display customer-wise total orders and total purchase amount.**

```
SELECT c.customer_name, COUNT(o.order_id) AS total_orders, SUM(o.amount) AS total_amount
FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name;
```

| customer_name | total_orders | total_amount |
|---------------|--------------|--------------|
| Ravi          | 2            | 8000.00      |
| Divya         | 2            | 16000.00     |
| Janu          | 1            | 12000.00     |

**Q15. Find customers who placed more than 2 orders and spent more than 8,000.**

```
SELECT c.customer_name, COUNT(o.order_id) AS total_orders, SUM(o.amount) AS total_amount
FROM Customers2 c JOIN Orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name
HAVING COUNT(o.order_id) > 2 AND SUM(o.amount) > 8000;
```

## STUDENTS TABLE

**Q1. Find the average marks scored by students in each department.**

```
SELECT department, AVG(marks) AS avg_marks FROM Students GROUP BY department;
```

| department | avg_marks |
|------------|-----------|
| CSE        | 88.5000   |
| ECE        | 78.0000   |
| IT         | 65.0000   |

**Q2. Find departments whose average marks are above 75.**

```
SELECT department, AVG(marks) AS avg_marks FROM Students GROUP BY department HAVING
AVG(marks) > 75;
```

| department | avg_marks |
|------------|-----------|
| CSE        | 88.5000   |
| ECE        | 78.0000   |

**Q3. Find the highest mark scored in each department.**

```
SELECT department, MAX(marks) AS highest_mark FROM Students GROUP BY department;
```

| department | highest_mark |
|------------|--------------|
| CSE        | 92           |
| ECE        | 78           |
| IT         | 65           |

**Q4. Find the total number of students in each department.**

```
SELECT department, COUNT(*) AS total_students FROM Students GROUP BY department;
```

| department | total_students |
|------------|----------------|
| CSE        | 2              |
| ECE        | 1              |
| IT         | 1              |

**Q5. Find departments having more than 5 students.**

```
SELECT department, COUNT(*) AS total_students FROM Students GROUP BY department HAVING
COUNT(*) > 5;
```

**Q6. Display departments sorted by average marks in descending order.**

```
SELECT department, AVG(marks) AS avg_marks FROM Students GROUP BY department ORDER BY
avg_marks DESC;
```

| department | avg_marks |
|------------|-----------|
| CSE        | 88.5000   |
| ECE        | 78.0000   |
| IT         | 65.0000   |

**Q7. Find the top 3 departments based on average marks.**

```
SELECT department, AVG(marks) AS avg_marks FROM Students GROUP BY department ORDER BY avg_marks DESC LIMIT 3;
```

| department | avg_marks |
|------------|-----------|
| CSE        | 88.5000   |
| ECE        | 78.0000   |
| IT         | 65.0000   |

**Q8. Find departments whose average marks are between 70 and 90.**

```
SELECT department, AVG(marks) AS avg_marks FROM Students GROUP BY department HAVING AVG(marks) BETWEEN 70 AND 90;
```

| department | avg_marks |
|------------|-----------|
| CSE        | 88.5000   |
| ECE        | 78.0000   |

**Q9. Find the total marks scored by students in each department.**

```
SELECT department, SUM(marks) AS total_marks FROM Students GROUP BY department;
```

| department | total_marks |
|------------|-------------|
| CSE        | 177         |
| ECE        | 78          |
| IT         | 65          |

**Q10. Display departments sorted by the total number of students.**

```
SELECT department, COUNT(*) AS total_students FROM Students GROUP BY department ORDER BY total_students DESC;
```

| department | total_students |
|------------|----------------|
| CSE        | 2              |
| ECE        | 1              |
| IT         | 1              |

**Q11. Find the lowest mark scored in each department.**

```
SELECT department, MIN(marks) AS lowest_mark FROM Students GROUP BY department;
```

| department | lowest_mark |
|------------|-------------|
| CSE        | 85          |
| ECE        | 78          |
| IT         | 65          |

**Q12. Find departments where the highest mark is greater than 90.**

```
SELECT department, MAX(marks) AS highest_mark FROM Students GROUP BY department HAVING MAX(marks) > 90;
```

| department | highest_mark |
|------------|--------------|
| CSE        | 92           |

**Q13. Find the number of students scoring above 80 in each department.**

```
SELECT department, COUNT(*) AS students_above_80 FROM Students WHERE marks > 80 GROUP BY department;
```

| department | students_above_80 |
|------------|-------------------|
| CSE        | 2                 |

**Q14. Find departments where more than 3 students scored above 75.**

```
SELECT department, COUNT(*) AS students_above_75 FROM Students WHERE marks > 75 GROUP BY department HAVING COUNT(*) > 3;
```

**Q15. Display departments ordered by highest mark in descending order.**

```
SELECT department, MAX(marks) AS highest_mark FROM Students GROUP BY department ORDER BY highest_mark DESC;
```

| department | highest_mark |
|------------|--------------|
| CSE        | 92           |
| ECE        | 78           |
| IT         | 65           |